ELI — Memory

ELI v2.5.34

November 2026

Contents

ELI Memory Subsystem **1**

The two layers . . . 1
Files . . . 2
The storage policy (`policy.py`) . . . 2
The `Memory` class (`memory.py`) . . . 4
Schema . . . 5
Shared turn retrieval (`retrieval.py`) . . . 5
`recall_memory`: the hybrid retriever . . . 6
Document index (`document_index.py`) . . . 6
Vector store (`vector_store.py`) . . . 7
Knowledge graph (`knowledge_graph.py`) . . . 7
Truth layer (`memory_truth.py`) . . . 8
Promotion across tiers . . . 8
Measuring it . . . 8
Honest assessment . . . 8
Recent-history window . . . 9

ELI Memory Subsystem

> **Updated for v2.4.84.** Memory is governed by a storage policy (`eli/memory/policy.py`): every row has an origin, repeats are merged, weight follows a forgetting curve reinforced by use, and faded derived rows are archived. Recall applies a time window before ranking, and reports what it did. Turn retrieval is shared in `eli/memory/retrieval.py`; FAISS deletes use tombstones.

`eli/memory/` holds 8,996 lines in 11 files: relational, full-text, vector and graph storage, all local SQLite and FAISS. Companion to `project_overview.md`.

The two layers

ELI's memory is not "a dump of every interaction". There are two layers:

1. **Conversation history** (`conversation_turns`, `conversations`, `session_summaries`): broad and chronological. A turn is kept unless the persistence gate (`runtime/persistence_gate.py`) rejects it as junk. This is what a question about a period of time reads.
2. **Distilled memory** (`memories`, `semantic`, `user_patterns`, the knowledge graph): a smaller set that the gate and the storage policy judged worth keeping, labelled by origin, merged when repeated, and faded when nothing reinforces it. Faded derived rows move to `memories_archive` instead of being deleted.

The memory-runtime report states this explicitly (`EXPLAIN_MEMORY_RUNTIME`, memory internals), with live counts for each layer.

Files

File	Lines	Role
`memory.py`	6,034	the `Memory` class (85 public methods), schema, `DBPaths`, upkeep, module facade
`policy.py`	128	pure storage-policy functions (origin, dedupe key, strength, archive, merge)
`retrieval.py`	310	shared turn retrieval (`retrieve_for_turn`), turn cache, time window (turns and executed actions)
`claims.py`	266	dated claims about the user: valid-from and valid-to, learned-at, supersession
`unified_retrieval.py`	371	the orchestrator stages consume `retrieve_for_turn` through it; formats the verified-memory block and the period log
`document_index.py`	706	every document ELI has read, stored whole and searched by passage
`vector_store.py`	677	FAISS index, embedder, tombstones
`knowledge_graph.py`	703	entity and relation graph
`system_index.py`	278	indexed apps, executables, files
`memory_truth.py`	188	read-only inspection used by status surfaces
`memory_adapter.py`	131	compatibility adapter
`__init__.py`	278	facade (`get_memory`, `get_agent_memory`, path helpers)

The storage policy (`policy.py`)

Every stored row gets an **origin** from `classify_origin(source, kind, tags, text)`:

Origin	Meaning	Factor on half-life
`user_said`	the owner said it or confirmed it	1.0
`eli_said`	ELI's own assistant text	0.5
`tool`	executor or observation output	0.35
`news`	news digests and reflections	0.25
`telemetry`	bookkeeping (`is_bookkeeping_memory`)	0.15

Rules, all in `policy.py` and pinned by `tests/test_memory_policy.py`:

- **Dedupe.** `text_key` is a sha1 of the normalised text (first 20 hex characters). A repeat bumps `seen_count` and `last_seen` instead of adding a row; `merge_group` keeps the earliest event time.
- **Strength.** `2 ** (-days_since_last_touch / half_life)`, floored at `MIN_WEIGHT` (0.05). The half-life is the base half-life × origin factor × `(1 + 3 × importance)` × `(1 + 0.6 × ln(1 + uses))`, where uses are repeat sightings plus recalls.
- **Base half-life.** 30 days, scaled by the square root of the median gap between days the owner used ELI, clamped to 14–120 days (`adaptive_half_life_days`).
- **Pinning.** A `user_said` row with importance ≥ 0.85 stays at strength 1.0.
- **Archiving.** `should_archive` is true only for a derived (not `user_said`) row that has never been recalled, has importance < 0.85, and whose weight is ≤ 0.12.
- **Vector index.** Telemetry rows are not embedded (`wants_vector_index`).
- **Evidence about the user.** Only `user_said` counts (`counts_as_evidence_about_user`).

Columns added to `memories`: `event_ts`, `seen_count`, `last_seen`, `last_recalled`, `recall_count`, `origin`, `text_key`. Tables added: `memories_archive`, `memory_meta`. The `semantic` table gained `evidence_count`, `last_seen`, `evidence`.

What counts as the same fact, and what strengthens a memory

- **Dedupe key.** `normalise_text` lower-cases, single-spaces and trims sentence punctuation from the edge of each word, and keeps meaningful marks. "I love C++", "I love C#" and "I love C" are three facts, "balance -5" is not "balance 5", and "3.5" is not "35". "Not fine!" and "not fine" are one. A key-version marker makes upkeep recompute old keys once (`rekey_text_keys`); it never merges.
- **Retrieval is exposure, not use.** Recalling a memory only increments `exposure_count`. Its strength is reinforced by `record_recall_outcome(ids, helped=True)`, which the engine calls when the next user message does not correct the answer those memories went into; a correction lowers the weight and increments `corrected_count`. A wrong memory that keeps being retrieved cannot make itself stronger. `recall_count` now means answers a memory helped with, and only that protects a row from archiving.
- **What enters durable memory.** `persistence_gate.should_store_memory_text` rejects filler and reactions ("haha", "lol that is funny", "hello there"), text under four words, questions, and ELI's own internal output. Conversation turns are still logged in full; only durable memory is filtered.

Dated claims (`claims.py`)

`memory_claims` holds what ELI believes about the user as bitemporal rows: `valid_from`/`valid_to` (when it was true) and `recorded_at`/`superseded_at` (when ELI knew it). A first-person statement such as "I work nights now", "I moved to Berlin", "I work at Acme Corp" or "my dog is called Max" becomes a claim when it is stored. A single-valued relation supersedes the old value and keeps it as history, "I used to work nights" is recorded as history only, "I no longer work nights" retires the standing claim, and the same value again confirms instead of duplicating. `Memory.claims_current()`, `claims_during(start, end)`, `claims_known_at(when)` and `claim_history(relation)` answer "what is true now", "what held in spring" and "what did you believe on 1 March". Retrieval adds the relevant claims to the evidence (a period question gets what held then), and deleting the source memory withdraws its claims. Extraction is a small exact set of patterns, not a model call.

What the user is doing now is a claim too: `watching`, `playing`, `reading` and `listening_to`, from "I'm watching X", "still watching X", "I've been playing X" or "X is on in the background". They are read from every message the user sends, not only the ones kept as memories, and from what ELI sees playing (a player title with a season and episode, "Game of Thrones — S2 E5", is `watching` with origin `observed`). These change often, so the newest value replaces the old one whoever reported it, and a value that is the other's initials ("GOT" for "Game of Thrones") confirms it. A statement older than the standing claim, as when history is read again, is kept as history and does not replace it. Questions match a claim by its relation first ("what am I watching" finds `watching`), ignoring filler words, newest first.

Forgetting, lineage and conflicts

- **Forget.** "forget that my locker is 212" lists the matching memories and asks; on yes (or `confirm forget memories 12 15`) `Memory.forget(ids)` deletes them and everything derived from them: derived rows through `memory_lineage`, index entries and vector tombstones, claims, recall log rows, semantic-fact quotes, session summaries that carry the same content, profile items and briefs, stances and belief revisions, graph entities created with the memory, and the conversation turns. It returns a report of what was removed.
- **Lineage.** `link_derivation(child, parents)` records that a summary or reflection was made from other memories; `lineage_root` and `independent_sources` mean a fact and its summaries count as one source. A claim's confirmations count distinct roots only.
- **Conflicts.** A claim that is not the user's own word and contradicts one that is stands beside it as `disputed` (both marked contested) instead of replacing it. `claim_conflicts()` lists them, `claims.conflict_question` phrases the question, and `resolve_claim_conflict` records the answer. Hedged statements ("maybe", "thinking of", "if I") are never claims.
- **Retrieval scope.** `retrieve_for_turn` returns `searched`: live memory, exact codes and quoted phrases (looked up as written, so an embedding cannot blur "ORCHID-7319"), dated claims, and the archive, which is searched only when live evidence is thin and its hits are labelled. The diagnostics block states that scope, so "nothing stored" is only claimed for what was searched.

Daily upkeep

`Memory.run_upkeep()` runs at most once a day (`upkeep_due()`), started asynchronously at engine start and after responses (`upkeep_async()`), and skipped under `ELI_TEST_MODE`. Steps:

1. **backfill**: label existing rows with origin, key and event time (after copying the database to a `.pre_policy.bak` file the first time). A dry run reports what it would fill and writes nothing; 1a. **rekey**: recompute dedupe keys once after the key definition changes; 1b. **claims**: read the whole conversation again for claims, oldest first, once per extraction version (`claims.EXTRACT_VERSION`); a version not yet read makes upkeep due on the next start;
2. **consolidate**: merge exact duplicates and rebuild `memories_fts`;
3. **decay**: recompute `weight` with the policy strength (idempotent: a pure function of age, importance and use, so a missed run cannot leave a row over-weighted);
4. **archive**: move faded derived rows to `memories_archive` (`search_archive`, `restore_from_archive` bring them back);
5. **identity**: merge legacy user ids into one stable owner id (`owner_id`, kept in `memory_meta`); only UUID-shaped legacy ids are merged;
6. **learning**: remove repeated identical belief revisions, cap corroboration at the number of days the owner has used ELI, and drop ledger copies of replay rows;
7. **semantic**: fold repeated facts into one row with `evidence_count` and up to five quotes;
8. **documents**: document text the old folder reader filed as memories (`kind pdf:<id>`, from a tool) leaves memory and its files go to the document index; graph relations made from tool text go with it (`refile_documents`). Rows like these make upkeep due on the next start. One 349-PDF folder had made 2,399 of 2,929 memories and 969 relations such as "User prefers screening". Not `forget()`: that matches words across the whole profile, and book text shares words with everything.

A run only marks the day done when every step succeeded; otherwise `run_upkeep` reports `status: partial` with the failed steps, and upkeep retries after an hour. `integrity_report()` reports the state of all of this for the health checks: `ok` is false when vectors are missing, and `status` is `healthy` or `degraded`.

The `Memory` class (`memory.py`)

A single class that owns most persistent concerns:

- **Memory**: `store_memory`, `add_memory`, `recall_memory`, `search_memory`, `search_memories`, `memories_between`, `get_recent_memories`, `adjust_weight`, `apply_weight_decay`, `consolidate_memories`,

archive_faded, search_archive, restore_from_archive, store_episodic, store_semantic, recall_semantic, store_reflective.

- **Conversation**: add_conversation_turn, store_conversation, get_conversation_history, get_recent_conversatio (with since, until, role), get_recent_turns_since, search_conversations, get_turns_for_day, save_session_summary, get_session_summaries.
- **Habits**: log_habit_event, get_habit_events, add_habit_rule, get_habit_rules, get_detected_habits, record_habit_run. log_habit_event writes from the engine's _learn_from_result (2026-09-30: gated on source — a habit-scheduler-fired turn doesn't write here, so ELI's own automation can't reinforce itself as fresh evidence of a routine when detect_habits later mines this table for recurrence).
- **Self-improvement and learning**: log_learning_event, log_failure, log_correction, add_observation, log_improvement, add_capability_proposal, propose_capability, get_pending_proposals, get_recent_failures, get_recent_improvements, get_recent_observations. These tables are written on the **agent** database (SelfImprovementEngine opens get_agent_memory()); the user database holds empty copies.
- **Health**: get_stats, get_dashboard_counts, get_db_routing_info, integrity_report.

vector_store is a lazy property; the knowledge graph is reached through knowledge_graph.get_knowledge_graph().

Schema

The blank user.sqlite3 template holds 27 tables and three FTS5 indexes. memories (+ memories_fts), memories_archive, memory_meta, semantic, conversation_turns, conversations, session_summaries, kg_entities (+ FTS), kg_relations, recall_log, runtime_events, learning_replay, observations, habits, habit_events, habit_rules, user_patterns, user_model, eli_stances, belief_revisions, corrections, failures, error_tracking, improvements, capability_proposals, news_articles (+ FTS), news_reflections. agent.sqlite3 holds agent_dispatches, agent_metrics and the self-improvement tables.

emotion_events is owned by cognition/emotion_timeline.py, not the Memory class: it opens user.sqlite3 directly and creates its table on first use. Columns: ts, user_id, session_id, detected, expressed, valence (negative, positive or neutral), confidence, source, arousal, user_text and eli_prior_action (the action ELI ran on the preceding turn, so it can ask whether the mood turned because of something it did). Reads come back ORDER BY ts DESC, id DESC; the id tiebreak matters because several reads can land within one second.

Each conversation turn is stored in conversation_turns. A learning_replay row is also written per turn (it carries action, outcome and reward), and the evidence ledger records the turn once in runtime_events; replay rows are not mirrored into the ledger.

Shared turn retrieval (retrieval.py)

The single owner of semantic and conversation recall for a turn. BusMemoryAgent and the orchestrator both call retrieve_for_turn(), so a query is not searched twice with different budgets.

- **Turn cache**: 8 seconds per process. The key includes session, user, query, time window and every limit (semantic_limit, conv_limit, recent_limit, summary_limit, hop2_limit, merge_cap, verified_only), so a deeper pass never gets the shallower result. invalidate_turn_cache() clears it.
- **Time window first.** When the planner finds a period in the question (query_planner.parse_window), memories dated inside it are fetched by date (Memory.memories_between, ordered by importance × weight) and merged with the topic hits, hits outside the window are dropped, and the user's turns inside the window are read from conversation_turns, and what ELI did in the window is read from the evidence ledger (evidence_ledger.actions_between: every executed action with the executor's own result text, scoped to the asking user). The result carries window_stats (candidates, added by date, in window, turns, actions) and the orchestrator logs and reports it (memory_diag).
- **The period log.** unified_retrieval.format_period_log renders the window's turns and actions as one dated block ("What happened in that period"), in every mode including Quick. The actions the

question is about (media, apps, web, timers, files) are listed first with a digest of distinct tracks; when the period is larger than the budget the selection is balanced across its days and a per-day tally says what was left out. A follow-up with no period of its own ("check again in full") keeps the previous turn's period for 15 minutes.
- **Vector hits are re-checked.** A FAISS hit is used only if its memory row still exists and passes the same origin and verification filters as the SQL channels; telemetry rows are not embedded, and tombstones apply to either id field an entry carries.
- **Only questions get a window.** `plan_window` applies a period only to a question or recall request, so a statement that says "last night" or "today" cannot narrow retrieval to that period.
- **Explicit dates.** `parse_window` reads ISO dates ("2026-03-03"), "3 March", "March 3rd 2025", ranges ("between 1 May and 10 May"), "in June 2025" and "during 2024" as well as relative periods. A date with no year is its latest past occurrence.
- **Token budget.** `context_budget.chars_per_token()` measures the loaded model's own tokenizer and falls back to 3.5 characters per token.
- **Hop-2 deepening** when the first hits are sparse; **heuristic rerank** via `rerank_candidates()`; **contradiction detection** through the bus.
- **Dated evidence.** Each recalled line carries its event date (`evidence_format`); working memory keeps the original date rather than the date it was re-pinned.

`recall_memory`: the hybrid retriever

The primitive `retrieve_for_turn()` builds on:

1. **FAISS and FTS5 both run** and are fused by reciprocal rank (`fuse_ranked_lists`); a LIKE scan covers the case where FTS returns nothing. `keyword_only=True` skips FAISS. FTS5 is asked about the `text` column only: the tokenizer splits tags at _, so "memory" used to match every row tagged `memory_recall` or `working_memory`, whatever it said.
2. **Noise filtering** (`memory_exclusion_sql` in `core/self_provenance.py`): ELI's own bookkeeping kinds and sources never resurface as recalled user memories.
3. **Importance-weighted ordering** via `COALESCE(importance, 0.5)`.
4. **Reinforcement.** A recalled row gets `last_recalled` and `recall_count` updated and its weight reset to 1.0; the recall is written to `recall_log` with its `memory_id`.

Column detection (`_memory_table_columns`) guards against schema drift between versions.

Document index (`document_index.py`)

What ELI has read, kept whole. The document reader hands a reply the first 8,000 characters of the first 20 pages; the index stores the full text, so a question about page 40 has something to be answered from.

- **One SQLite file** (`documents.sqlite3`, `paths.document_index_db_path`): `documents`, `passages` (text and a float16 vector each) and an FTS5 table kept in step by triggers. Removing a document cascades to every passage and vector made from it.
- **What gets indexed.** Anything an action reads, summarises or writes (`READ_FILE`, `SUMMARIZE_FILE`, `ANALYZE_PDF`, `CREATE_DOCUMENT`, `DOC_GENERATE`, `GENERATE_DOCUMENT`, `CONVERT_DOCUMENT`, `WRITE_NOTE`, `NEW_NOTE`; hooked in the executor's post-dispatch), a file handed over with a message (`[File: path]`), and ELI's own `documents/` and `notes/` folders on first use. PDF, DOCX, ODT, EPUB and prose text formats only; names that look like secrets (`.env`, keys, credentials, wallets) and files over 96 MB are never stored. An unchanged file is not stored twice; a changed one replaces its passages.
- **How.** A worker thread extracts the whole text (`document_reader.document_sections`, with page numbers for PDFs), cuts it into ~900-character passages that overlap and start on a word, and embeds them with the embedder the memory store already loads. Embedding waits while a conversation turn is live. Wording search works as soon as the text is stored.
- **Search** (`search`): FTS5 BM25 over the question's content words and cosine over the vectors, fused by reciprocal rank. A question that names a document stays inside that document: two words of its title, or one when the question is about a document or a pack ("the garden guide"). One title

word said in passing no longer pulls the document in; "that file" means the one read last; a file attached to the message is stored before the search and the search stays inside it. Asked nothing about documents (`strict`), a passage is offered only at cosine >= 0.70, or >= 0.64 with at least half the question's content words in it (thresholds measured on the nomic embedder, where unrelated text scores up to 0.66).

- **A file or folder named by its path is handed over too.** A path typed into a message, or into one of the user's last messages when this one speaks of "the paths", "those folders" or "the project" (`named_paths`, `with_what_it_refers_to`), is looked at on the disk before the model is asked. The turn's context gets what is there (`describe_named`: how many files, of what kinds, in which subfolders, the newest by name; sized by `cog.named_paths_chars`, and what did not fit is counted). The documents in a named folder are stored the way a dropped file is, newest first, for as long as the memory agent is allowed for its own work; the rest are queued and read behind the reply. Given two folders to study, ELI had answered "that is a string literal to me" and asked the user to run `ls` and paste the output.
- **Scope.** Documents carry the user id of the turn that read them; another API user never sees them. Indexing and removal are written to the evidence ledger.
- **Forgetting.** "forget the X document" lists matching documents and, on "confirm forget documents", removes them from the index. The file on disk is untouched.
- **Off switch.** `ELI_DOCUMENT_INDEX=0`.

Vector store (`vector_store.py`)

FAISS `IndexFlat`, embeddings from a local nomic embedder (llama.cpp). Where the embedder runs (`embedder_placement`): on the GPU with the most room, discrete or sharing the machine's memory, when it takes its place before the language model is planned; the model loader asks the store to be `ready()` first, so the plan (`core/load_plan.py`) sees what it holds. Measured on one machine: on a GPU it embeds a memory in about 4 ms for about 130 MB of the card, in RAM in over a second while the processor is busy. In RAM, every part of it (`n_gpu_layers=0`, `offload_kqv` and `op_offload` off where the runtime has them, `embedder_in_ram`): with no GPU, once the language model is loaded, or when a GPU will not take it. With only its layers on the processor, llama.cpp had still put its working buffers on the GPU (54 MB on a CUDA build) after the plan had left the card a sliver, and the next reply aborted.

- `_embed_lock` (RLock) serialises embedding: the embedder is not thread-safe, which is why orchestrator retrieval is sequential.
- Metadata lives in `meta.json` (migrated from a legacy `meta.pkl`).
- **Saves are batched.** Each save rewrites `index.faiss` and `meta.json` whole (18 MB at 2,900 memories), so `add()` saves every `SAVE_EVERY` adds on a background thread and shutdown saves the rest (`flush(pending_only=True)`). `store_memory` used to save after every row: a 349-PDF folder wrote 24 GB in 50 minutes and stalled the whole machine on disk. At start the index adds whatever memories it is missing (an unclean exit), or rebuilds when it is empty or its entries carry no memory ids.
- Singleton via `get_vector_store()`; shutdown-aware; `reset_vector_store()` for rebuilds.
- **Tombstones:** `mark_memory_deleted(id)` writes to `meta.tombstones.json` next to the metadata; search skips tombstoned rows without a rebuild; `compact_tombstones(live_ids)` reclaims space.
- `store_memory()` returns `vector_indexed: bool`, a failed index write logs at warning level, and `fire_memory_uncertainty_event()` drives the World tab's `memory_uncertainty` bar.
- Tombstoning is a per-call-site discipline: there is no shared `delete_memory_row()` helper, so any direct `DELETE FROM memories` must tombstone the vector itself (`consolidate_memories` and `profile_extractor._scrub_onboarding_snapshot` do).

Knowledge graph (`knowledge_graph.py`)

`kg_entities(name, type, aliases, description, confidence)` and `kg_relations(subject_id, predicate, object_id, weight, source)`: a subject-predicate-object graph. FTS5 over entities with insert, update and delete triggers for fuzzy `search_entities`. Only what the user said feeds it (`feeds_graph`, the memory policy's evidence about the user): the extractor reads text as the user's own words, so document and tool text made "User prefers screening". `rebuild_from_memories` is the one rebuild, used by the self-upgrade

and settings buttons. `upsert_entity`; `context_for_prompt` gives lightweight SQLite-only prompt context with no embedding. Relations an older extractor stored that are not facts (an is_a that is a mood or an activity, a has_name that is not a name; `_junk_name_relation`) are left out from both sides, and an entity that had nothing else is left out with them. A stop-word list stops common words becoming entities. A relation's `ts` is when it was first learned: adding it again raises its weight and leaves the date alone.

Truth layer (`memory_truth.py`)

`inspect_sqlite` and `inspect_vector_store`: read-only inspection used by status surfaces; reads `vectors/meta.json` and falls back to the legacy pickle. Backs `truth_report` and the memory-status surfaces; `format_memory_truth` is the short answer to "how is your memory doing?" (`MEMORY_STATUS` with `memory_scope=health`).

A recall reply (`MEMORY_RECALL`) puts each memory's date in front of it (`turn_stamp`), so "when was that?" has an answer on the screen instead of one the model makes up.

Promotion across tiers

Promotion is a fan-out from `user_patterns`:

```
turns ──▶ user_patterns ──┬──▶ semantic   (profile_extractor._promote_to_semantic)
                          └──▶ KG entities and relations
                               (persona_updater._populate_kg_from_user_patterns)
```

`_promote_to_semantic` applies a durable/transient prefix filter, dedupes against `semantic` and fires only when `_insert_user_pattern` actually inserted, so reaffirmations do not re-promote. Corroboration counts days the owner reaffirmed a fact, not extraction passes, and a passing mention cannot overwrite a fact the owner stated and reaffirmed (`_insert_user_pattern` weighs provenance and corroboration; each supersession is recorded in `belief_revisions`, and repeats are deduplicated). `store_episodic` and `store_semantic` are thin aliases and are not the promotion path.

Measuring it

`tools/eval/memory_bench.py` runs 11 deterministic scenarios against a temporary database, with no model: extraction, multi-session recall, a relative and an explicit time window, knowledge update, abstention, provenance, deletion, no destructive merge, no self-reinforcement, and chatter not becoming durable knowledge. They follow the LongMemEval ability categories but are written for ELI; they are not the official dataset. `tests/test_memory_bench.py` runs them, so a memory change that regresses one fails the suite.

Honest assessment

- **Strong:** genuinely hybrid (vector, FTS5, graph) on one local SQLite foundation; a measurable storage policy with an audit trail; recall that filters by date before ranking; noise filtering that keeps ELI's own reflections out of recall; embedder serialisation that avoids the segfault.
- **Weak:**
  1. `memory.py` is a 6k-line class spanning unrelated concerns (semantic, conversation, habits, learning, failures, capabilities, upkeep). It wants splitting along those seams.
  2. **Schema sprawl.** `memories` and a legacy `memory`, `conversations` and `conversation_turns`, plus a standalone `semantic` table; inline column detection compensates for schema churn.
  3. **Narrow intake.** Promotion works, but it is fed by regex extraction plus one LLM pass per session, so `user_patterns`, the semantic tier and the knowledge graph stay small relative to the conversation history. Widening the extractor is the work.
  4. **Retrieval cannot detect nonsense.** Similarity bands for genuine and gibberish queries overlap, so no absolute similarity floor separates them; the relative cutoff in `vector_store` tightens the candidate pool but is not a relevance oracle.

5. **No shared delete-and-tombstone helper** (above).

`eli/cognition/reranker.py`'s heuristic reranker weighs overlap 0.45, importance 0.20, weight 0.15, recency 0.10. Importance and recency alone (0.45 combined) could match the overlap term's full weight, so a memory saved with "please remember this" (importance near 1.0) could outrank a memory that actually matches a vague query, on nothing but being generally important and recent — proven against a live database, where a stored test code beat memories that were actually about the asked-for week. The importance/weight/recency term is now gated: full credit only above a real overlap floor (0.2), a reduced share when a retriever found the candidate at all but overlap is weak or incidental (a single shared word in a long memory no longer counts as "on topic"), and the smallest share when neither applies. That gate had a blind spot: a date-range question ("what have we discussed the past 7 days") is answered largely by `memories_between()`, a separate lookup that matches by timestamp, not by the keyword/semantic retrievers, so its rows carry neither a lexical-overlap score nor the "a retriever found this" channel markers the gate checked for — it read that as "nobody vouched for this" and crushed those rows to a fifth of their score, exactly the rows a "past week" question needs most. Being inside the exact window asked about is now its own relevance signal, gated to full credit like a real overlap match. `tests/test_reranker_importance_does_not_override_relevance.py` pins both.

Recent-history window

The user-facing tunable `cog.mem_recent_turns` (Settings ▸ Cognition; default 24, maximum 80) controls how many recent turns enter the prompt. `RECENT_HISTORY_CAP` no longer exists (the old memory-evidence module was removed); the prompt assembler and `context_budget` do the budgeting, and a recall question keeps a floor of a third of the window for memory context.

The window holds this conversation only (`evidence_format.this_conversation`): turns older than 12 hours, or on the far side of a 3-hour silence, are left out of the dialogue block. Earlier days still reach the prompt, dated, through retrieval and the period log. Without this a two-day-old argument sat in the prompt as if it were still going on.